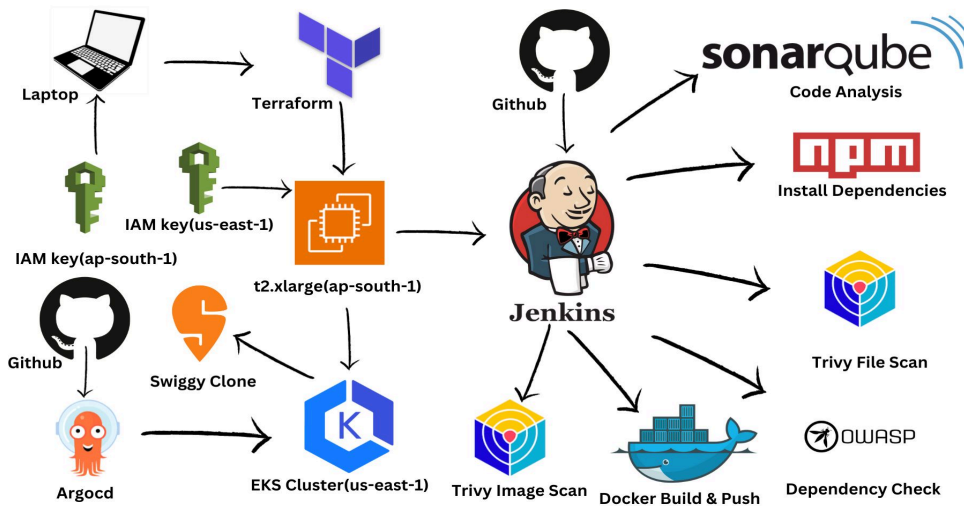




Dushyant Kumar's Blog

[Follow](#)

DevSecOps Project : Swiggy Clone 🍴🥗



Dushyant Kumar

Dec 31, 2023 • 📖 11 min read

Show more ▾



In today's fast-paced digital landscape, building and deploying applications not only requires speed but also airtight security. That's where DevSecOps comes into play, blending development, security, and operations into a single, unified process.

In this guide, we will start a journey where we'll leverage Terraform, Jenkins CI/CD, SonarQube, Trivy, ArgoCD, and Amazon Elastic Kubernetes Service (EKS) to create a robust and secure pipeline for deploying applications on Amazon Web Services (AWS).

Whether you're an experienced developer looking to enhance your DevSecOps skills or a newcomer eager to explore this exciting intersection of software development and security, this guide has something valuable to offer.

Let's start and explore the steps to safeguard your Amazon app while ensuring smooth and efficient deployment process.

Overview:

- **Infrastructure as Code:** Use Terraform to define and manage AWS infrastructure for the application.
- **Container Orchestration:** Utilize Amazon EKS for managing and scaling containerized applications.
- **CI/CD with Jenkins:** Set up Jenkins to automate building, testing, and deploying the application.
- **Static Code Analysis:** Incorporate SonarQube to analyze code quality and identify vulnerabilities.
- **Container Image Scanning:** Integrate Trivy to scan container images for security issues.
- **Application Deployment with ArgoCD:** Use ArgoCD for declarative, GitOps-based application deployment on EKS.

Project Resources:

GitHub Link:

[SWIGGY-CLONE-REPOSITORY](#) - Application code & Manifest files.

Step1: Create IAM Users

Navigate to the **AWS console**

- Create two IAM users with the following permissions.
- Create two IAM users with the following permissions.
- Create one t2.xlarge EC2 instance with Ubuntu ami using terraform.
- Login to the EC2 instance and follow the below steps.

📌 Step2: Aws Configuration

Install the AWS Cli in your local machine (windows laptop)

Check AWS CLI is install on your lapto:

COPY

```
aws --version
aws configure
```

Provide your Aws **Access key** and **Secret Access key**

```
DELL@DESKTOP-PLK6IAS MINGW64 ~/Desktop/swiggy/terraform_jenkins_serversetup
$ ls
advance_user_data.sh*  ec2_instance.tf  provider.tf      terraform.tf      variable.tf
dynamo_lock_table.tf  jenkins_sg.tf   s3_state_bucket.tf  terraform.tfvars

DELL@DESKTOP-PLK6IAS MINGW64 ~/Desktop/swiggy/terraform_jenkins_serversetup
$ aws configure
AWS Access Key ID [*****RQOV]: A[REDACTED]
AWS Secret Access Key [*****i50]: [REDACTED]LtqMsoC[REDACTED]
Default region name [ap-south-1]: ap-south-1
Default output format [json]:

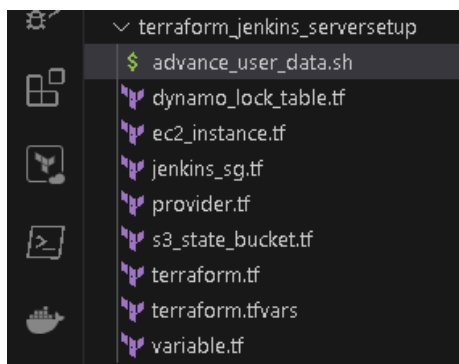
DELL@DESKTOP-PLK6IAS MINGW64 ~/Desktop/swiggy/terraform_jenkins_serversetup
$
```

📌 Step3: Terraform files and Provision Jenkins, sonarqube

Check terraform Installation in yor laptop :

COPY

```
terraform --version
```



Terraform Repository



This will install Jenkins, Docker, Sonarqube, and Trivy, kubectl, eksctl, aws cli by Terraform with an EC2 instance(t2.xlarge).

✦ Terraform commands to provision:

COPY

```
terraform init
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS bash

Initializing provider plugins...
- Finding hashicorp/aws versions matching "5.11.0"...
- Installing hashicorp/aws v5.11.0...
- Installed hashicorp/aws v5.11.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

COPY

```
terraform fmt
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS bash

DELL@DESKTOP-PLK6IAS MINGW64 ~/Desktop/swiggy/terraform_jenkins_serversetup
$ terraform fmt
ec2_instance.tf
terraform.tf
variable.tf
```

COPY

```
terraform validate
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS bash + v

DELL@DESKTOP-PLK6IAS MINGW64 ~/Desktop/swiggy/terraform_jenkins_serversetup
$ terraform validate
Success! The configuration is valid.

DELL@DESKTOP-PLK6IAS MINGW64 ~/Desktop/swiggy/terraform_jenkins_serversetup
$
```

COPY

```
terraform plan
```

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
DELL@DESKTOP-PLK6IAS MINGW64 ~/...
$ terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following
symbols:
+ create

Terraform will perform the following actions:

# aws_dynamodb_table.dynamo_state_table will be created
+ resource "aws_dynamodb_table" "dynamo_state_table" {
+   arn                = (known after apply)
+   billing_mode       = "PAY_PER_REQUEST"
+   hash_key           = "LockID"
+   id                 = (known after apply)
+   name               = "swiggy-dynamodb-table-2024"
+   read_capacity      = (known after apply)
+   stream_arn         = (known after apply)
+   stream_label       = (known after apply)
+   stream_view_type   = (known after apply)
+   tags               = {
+     "Name" = "swiggy-dynamodb-table-2024"
+   }
+   tags_all           = {

```

```

jenkins_sg.tf
provider.tf
s3_state_bucket.tf
terraform.tf
terraform.tfvars
variable.tf

} },
+ name                = "Jenkins-sg"
+ name_prefix         = (known after apply)
+ owner_id            = (known after apply)
+ revoke_rules_on_delete = false
+ tags               = {
+   "Name" = "Jenkins-security-group"
+ }
+ tags_all            = {
+   "Name" = "Jenkins-security-group"
+ }
+ vpc_id              = (known after apply)
}

Plan: 5 to add, 0 to change, 0 to destroy.

```

COPY

terraform apply

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
terraform + bash

+ self                = false
+ to_port             = 9000
},
}
+ name                = "Jenkins-sg"
+ name_prefix         = (known after apply)
+ owner_id            = (known after apply)
+ revoke_rules_on_delete = false
+ tags               = {
+   "Name" = "jenkins-security-group"
+ }
+ tags_all            = {
+   "Name" = "jenkins-security-group"
+ }
+ vpc_id              = (known after apply)
}

Plan: 5 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value:

```

```

terraform.tfvars  terraform.tf  ec2_instance.tf
terraform_jenkins_serversetup > terraform.tf > terraform

1 terraform {
2   required_providers {
3     aws = {
4       source = "hashicorp/aws"
5       version = "5.11.0"
6     }
7   }
8   # creating backend infra for state management
9   /* backend "s3" {
10    bucket = "s3-state-backend-12-2023"
11    key    = "terraform.tfstate"
12    region = "ap-south-1"
13    encrypt = true
14    dynamodb_table = "dynamo-state-locking-12-2023"
15  }*/
16 }

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
aws_instance.my_instance: Still creating... [10s elapsed]
aws_instance.my_instance: Still creating... [20s elapsed]
aws_instance.my_instance: Creation complete after 22s [id=i-01fc3f57e7d766742]

Apply complete! Resources: 5 added, 0 changed, 0 destroyed.

DELL@DESKTOP-PLK6IAS MINGW64 ~/Desktop/swiggy/terraform_jenkins_serversetup
$

```

Now uncomments th



```

terraform.tfvars  terraform.tf  ec2_instance.tf
terraform_jenkins_serversetup > terraform.tf > terraform > backend "s3" > dynamodb_table
1  terraform {
2      required_providers {
3          aws = {
4              source = "hashicorp/aws"
5              version = "5.11.0"
6          }
7      }
8      # creating backend infra for state management
9      backend "s3" {
10         bucket      = "swiggy-s3-bucket-2024"
11         key         = "terraform.tfstate"
12         region      = "ap-south-1"
13         encrypt     = true
14         dynamodb_table = "swiggy-dynamodb-table-2024"
15     }
16 }

```

COPY

terraform init

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
DELL@DESKTOP-PLK6IAS MINGW64 ~/Desktop/swiggy/terraform_jenkins_serversetup
$ terraform init

Initializing the backend...
Do you want to copy existing state to the new backend?
Pre-existing state was found while migrating the previous "local" backend to the
newly configured "s3" backend. No existing state was found in the newly
configured "s3" backend. Do you want to copy this state to the new "s3"
backend? Enter "yes" to copy and "no" to start with an empty state.

Enter a value: yes

Successfully configured the backend "s3"! Terraform will automatically
use this backend unless the backend configuration changes.

Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v5.11.0

Terraform has been successfully initialized!

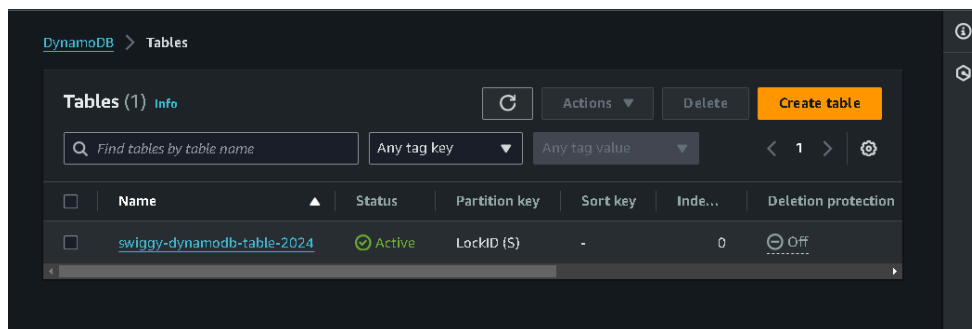
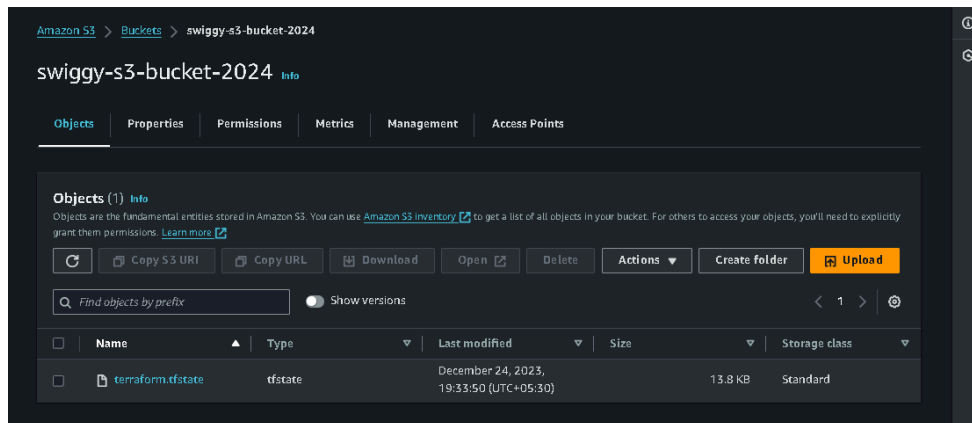
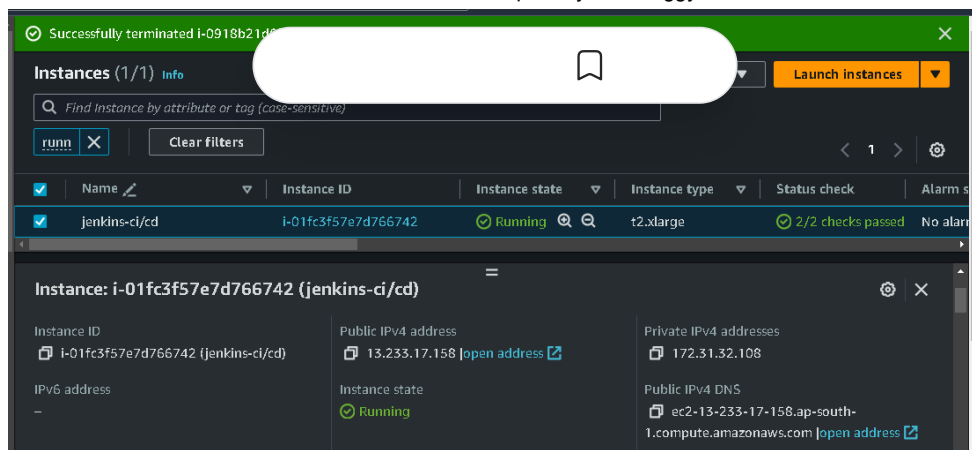
You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

```

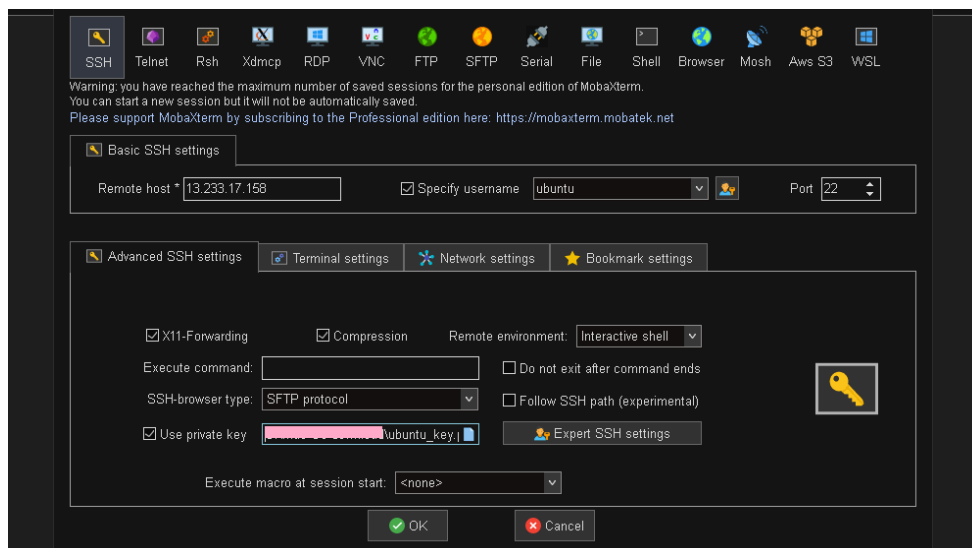
Output:

The EC2 instance Jenkins-sonarqube-trivy-vm is created by Terraform with Jenkins, Sonarqube, Trivy, kubectl, eksctl, aws cli as userdata for the EC2 instance, which is installed during the creation of the EC2 instance.

S3 bucket is created with terraform.tf statefile present in it and dynamodb table is created for state locking the terraform state.



SSH my ec2 instance



COPY

<Public IPV4 address>
13.233.17.158:8080



Getting Started

Unlock Jenkins

To ensure Jenkins is securely set up by the administrator, a password has been written to the log (not sure where to find it?) and this file on the server:

```
/var/lib/jenkins/secrets/initialAdminPassword
```

Please copy the password from either location and paste it below.

Administrator password

Continue

COPY

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

```
ubuntu@ip-172-31-32-108:~$ cat /var/lib/jenkins/secrets/initialAdminPassword
cat: /var/lib/jenkins/secrets/initialAdminPassword: Permission denied
ubuntu@ip-172-31-32-108:~$ sudo cat /var/lib/jenkins/secrets/initialAdminPassword
dbb72394c5c1425ab4d82714c79f80da
ubuntu@ip-172-31-32-108:~$ ^C
ubuntu@ip-172-31-32-108:~$
```

Unlock Jenkins using an administrative password and install the suggested plugins.

Getting Started

Customize Jenkins

Plugins extend Jenkins with additional features to support many different needs.

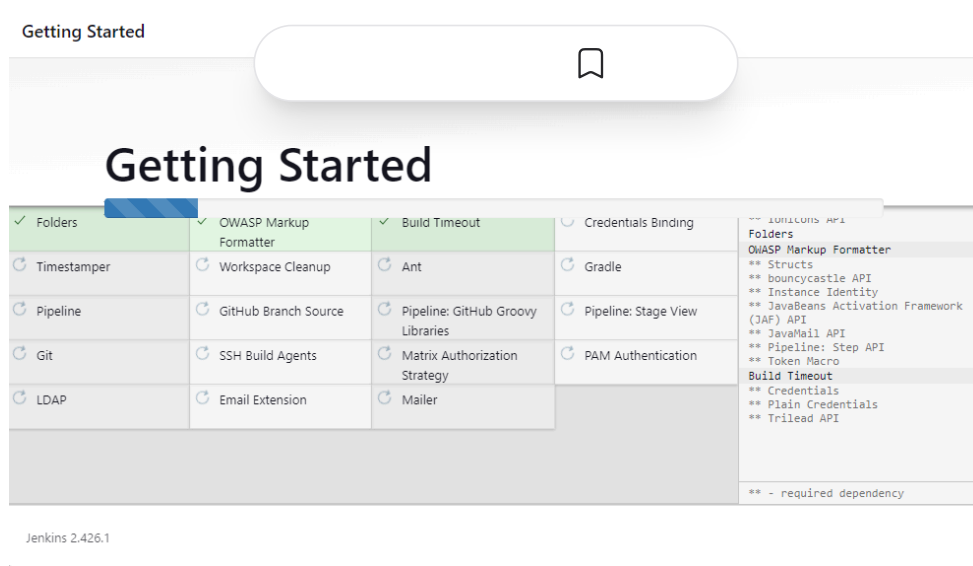
Install suggested plugins

Install plugins the Jenkins community finds most useful.

Select plugins to install

Select and install plugins most suitable for your needs.

Jenkins 2.426.1



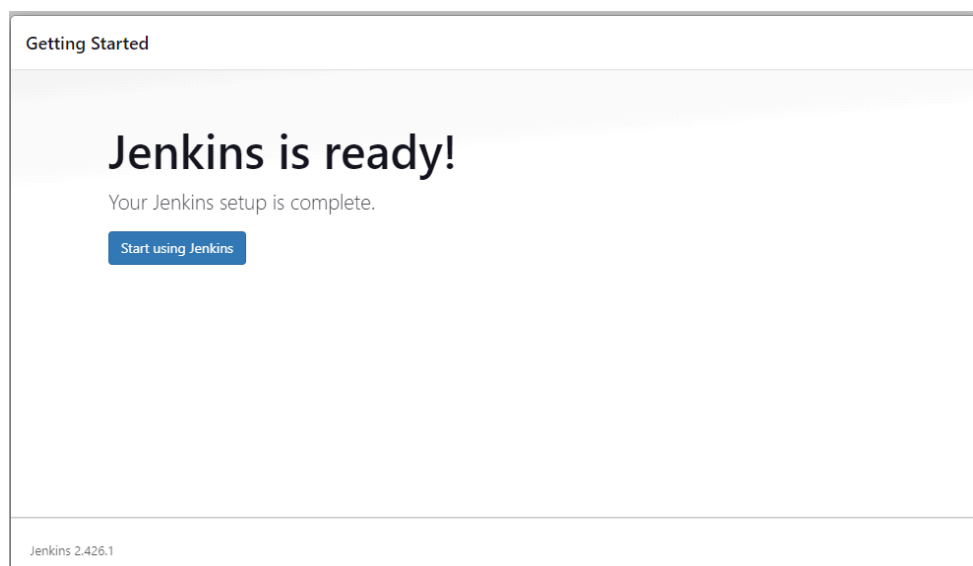
Jenkins will now get installed and install all the libraries.

The screenshot shows the 'Create First Admin User' form in Jenkins. The form has four input fields: Username, Password, Confirm password, and Full name. At the bottom right, there are two buttons: 'Skip and continue as admin' and 'Save and Continue'.

Jenkins 2.426.1

Create a user, click save, and continue.

Jenkins Getting Started Screen.





Welcome to Jenkins!

This page is where your Jenkins jobs will be displayed. To get started, you can set up distributed builds or start building a software project.

Start building your software project

Create a job



Set up a distributed build

Set up an agent



Configure a cloud



Learn more about distributed builds



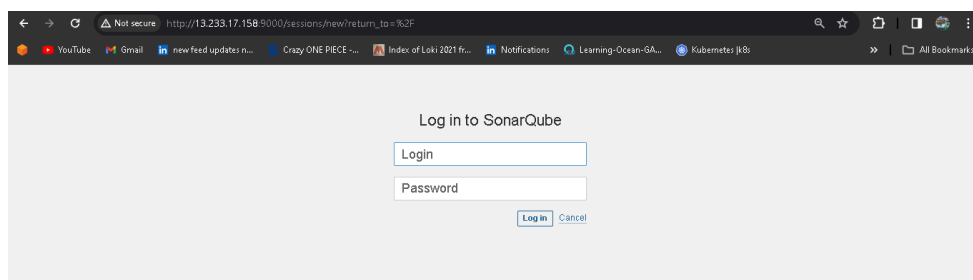
Copy your public key again and paste it into a new tab.

COPY

```
<instance-public-ip>:9000
13.233.17.158:9000
```

Now our Sonarqube is up and running as a Docker container.

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	N
AMES	sonarqube:lts-community	"/opt/sonarqube/dock..."	16 minutes ago	Up 16 minutes	0.0.0.0:9000→9000/tcp, :::9000→9000/tcp	s



Enter your username and password, click on login, and change your password.

COPY

```
# by default username and password is admin
username admin
password admin
```

Set new password for login.

Update your password

This account should not use the default password.

Enter a new password

All fields marked with * are required

Old Password *

.....

New Password *

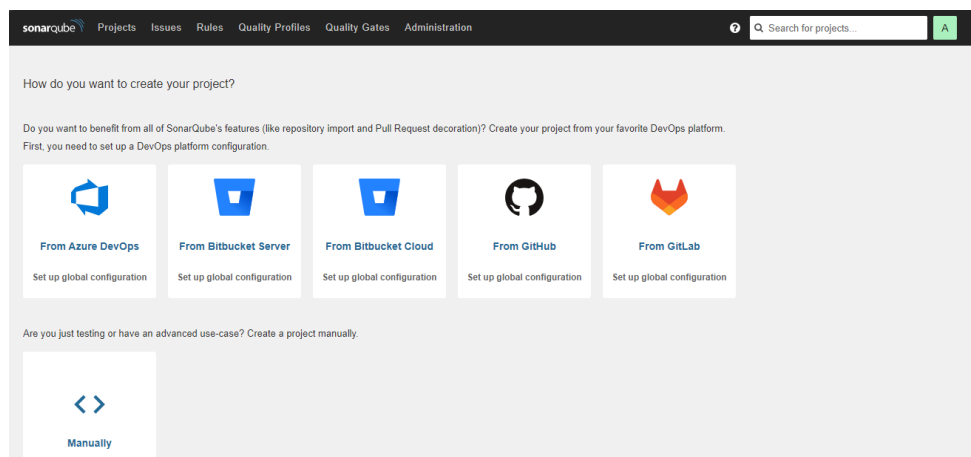
.....

Confirm Password *

.....

Update

Update the new password. This is the Sonar Dashboard, as shown below.



Check Trivy version

Check the Trivy version in an Ec2 instance.

COPY

```
trivy --version
```

Step 4: Install Plugins like JDK, Sonarqube Scanner, NodeJs, and OWASP Dependency Check

4A – Install Plugin

Goto Manage Jenkins → Plugins → Available Plugins

Install below plugins



1: Eclipse Temurin Installer (Install without restart)

2: SonarQube Scanner (Install without restart)

3: Sonar Quality Gates (Install Without restart)

4: NodeJs (Install Without restart)

Plugins

- Updates
- Available plugins
- Installed plugins
- Advanced settings
- Download progress

sonar quality gates		
Install	Name	Released
☒	Eclipse Temurin Installer 1.5 Provides an installer for the JDK tool that downloads the JDK from https://adoptium.net	1 yr 2 mo ago
☒	SonarQube Scanner 2.16.1 External Site/Tool Integrations Build Reports This plugin allows an easy integration of SonarQube , the open source platform for Continuous Inspection of code quality.	2 mo 15 days ago
☒	Sonar Quality Gates 1.3.1 Fails the build whenever the Quality Gates criteria in the Sonar 5.6+ analysis aren't met (the project Quality Gates status is different than "Passed"). Warning: This plugin version may not be safe to use. Please review the following security notices: Credentials transmitted in plain text	5 yr 4 mo ago

Plugins

- Updates
- Available plugins
- Installed plugins
- Advanced settings
- Download progress

node		
Install	Name	Released
☐	Role-based Authorization Strategy 693.v731678c3e0eb_ Security Authentication and User Management Enables user authorization using a Role-Based strategy. Roles can be defined globally or for particular jobs or nodes selected by regular expressions.	4 mo 2 days ago
☒	NodeJS 1.6.1 api NodeJS Plugin executes NodeJS script as a build step.	4 mo 9 days ago
☐	built-on-column 1.4 List view columns Shows the actual node that a job was built on.	8 mo 16 days ago
☐	Throttle Concurrent Builds 2.14 Build Wrappers Cluster Management Agent Management This plugin allows for throttling the number of concurrent builds of a project running per node or globally.	6 mo 6 days ago

4B: Configure Java and Nodejs in Global Tool Configuration

Goto Manage Jenkins → Tools → Install JDK (17) and NodeJs (16). Click on Apply and Save

Add JDK

JDK

Name

jdk17

☒ Install automatically ?

Install from adoptium.net ?

Version ?

jdk-17.0.8.1+1

Add Installer

Add JDK

NodeJS installations

Add NodeJS

NodeJS

Name
node16

☒ Install automatically ?

Install from nodejs.org

Version
NodeJS 21.4.0

For the underlying architecture, if available, force the installation of the 32bit package. Otherwise the build will fail

☐ Force 32bit architecture

Save Apply

Step 5: Configure Sonar Server in Manage Jenkins

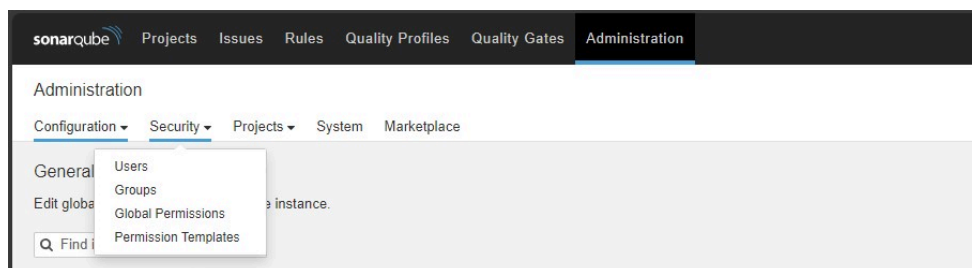
Grab the public IP address of your EC2 instance where sonar is running.

Sonarqube works on Port 9000, so <Public IP>:9000.

Go to your Sonarqube server.

5A: Create new token in SonarQube

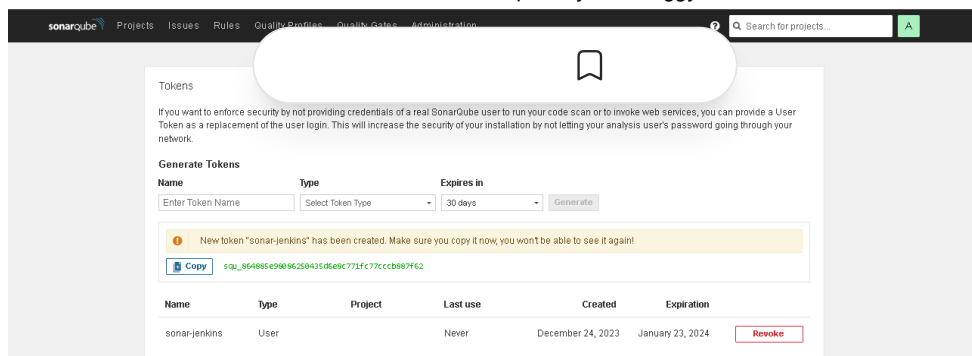
Click on **Administration** → **Security** → **Users** → Click on **Tokens** and **Update Token**, → Give it a name, and click on **Generate Token**



click on update Token



Create a token with a name and generate



copy this Token

5B: Configure SonarQube Credentials

Goto Jenkins Dashboard → Manage Jenkins → Credentials → Add secret text.

It should look like this

Security

- Security**
Secure Jenkins; define who is allowed to access/use the system.
- Credentials**
Configure credentials
- Credential Providers**
Configure the credential providers and types
- Users**
Create/delete/modify users that can log in to this Jenkins.

New credentials

Kind
Secret text

Scope ?
Global (Jenkins, nodes, items, all child items, etc)

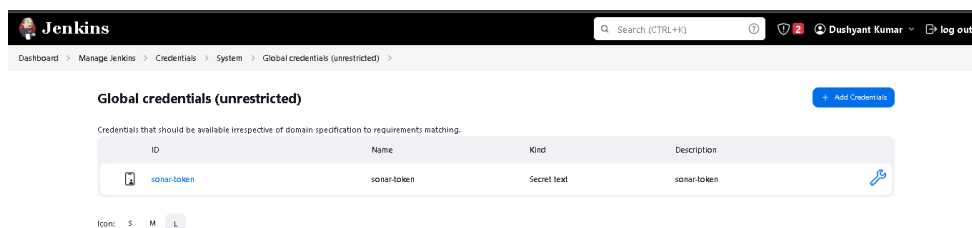
Secret

ID ?
sonar-token

Description ?
sonar-token

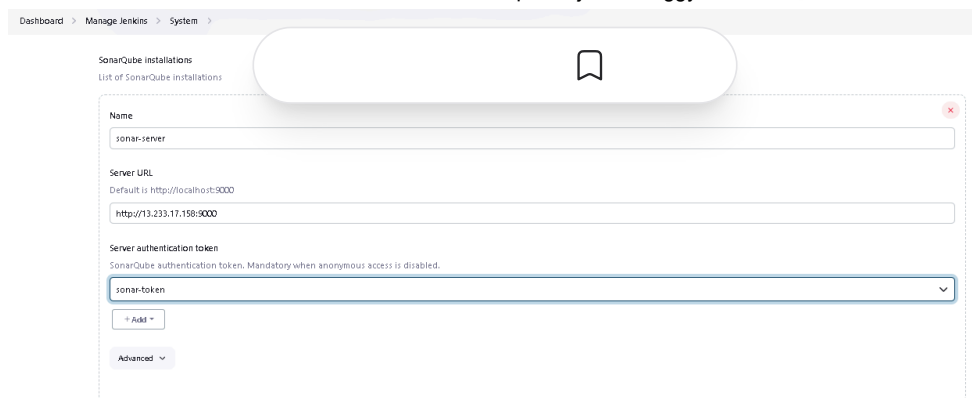
[Create](#)

You will see this page once you click on create



5C: Configure Sonar Server in Manage Jenkins

Now, go to Dashboard → Manage Jenkins → System and add like the below image.



Dashboard > Manage Jenkins > System

SonarQube installations
List of SonarQube installations

Name
sonar-server

Server URL
Default is `http://localhost:9000`
http://13.233.17.158:9000

Server authentication token
SonarQube authentication token. Mandatory when anonymous access is disabled.
sonar-token

+ Add

Advanced

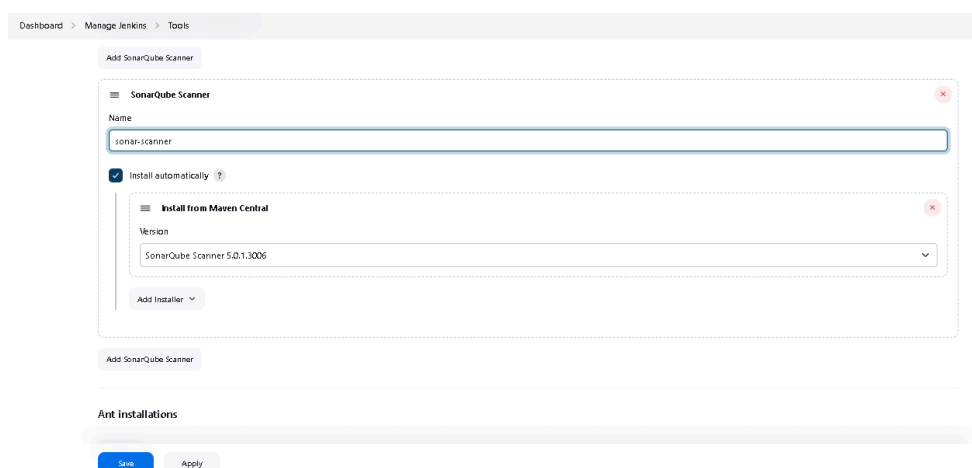
Click on Apply and Save.

The **Configure System** option is used in Jenkins to configure different server

Global Tool Configuration is used to configure different tools that we install using Plugins

🌟 5D: Configure Sonar Scanner in Manage Jenkins --> Tools

We will install a sonar scanner in the tools.



Dashboard > Manage Jenkins > Tools

Add SonarQube Scanner

SonarQube Scanner

Name
sonar-scanner

☒ Install automatically ?

Install from Maven Central

Version
SonarQube Scanner 5.0.1.3006

Add Installer

Add SonarQube Scanner

Ant installations

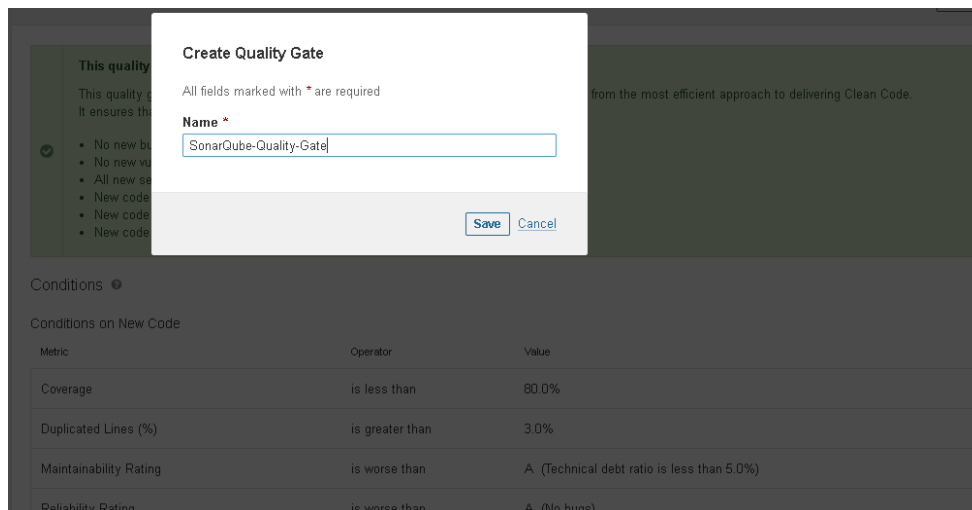
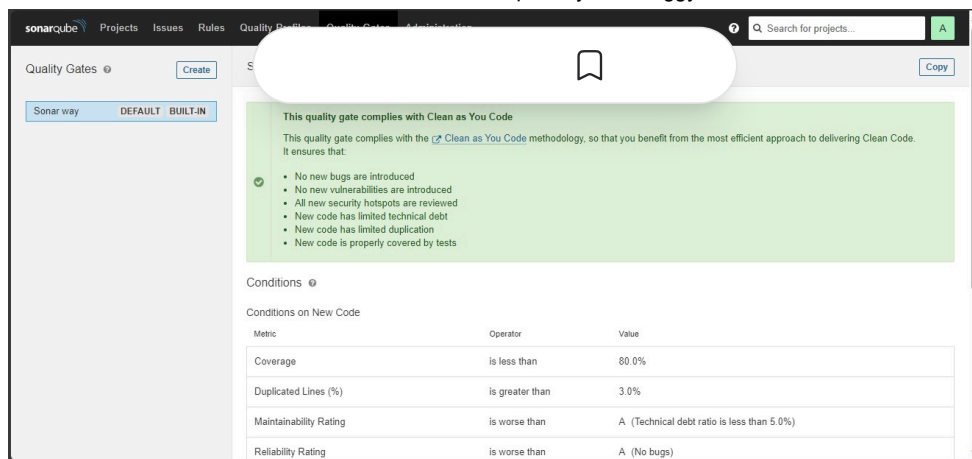
Save Apply

🌟 5E: Configure Quality gate in SonarQube Server

In the Sonarqube Dashboard, add a quality gate as well.

In the sonar interface, create the quality gate as shown below:

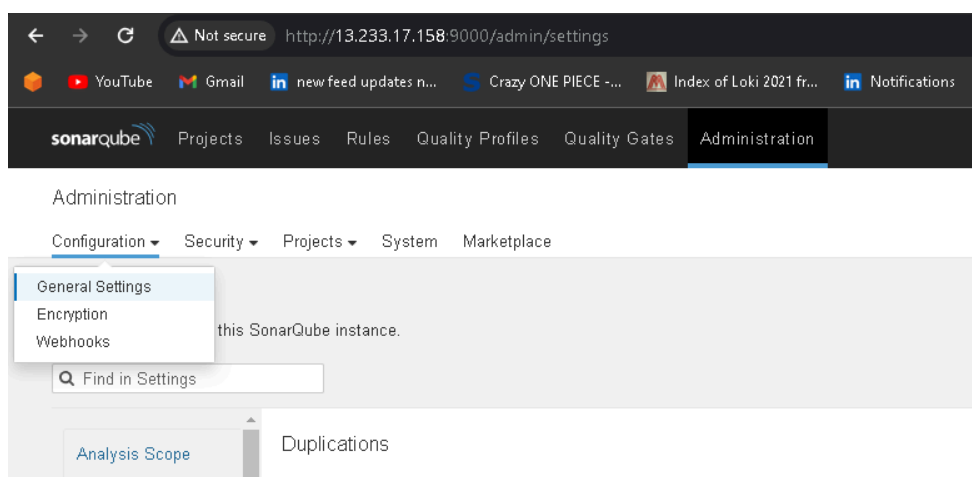
Click on the **quality gate**, then **create**.



Click on the **save** option.

In the Sonarqube Dashboard, Create **Webhook** option as shown in below:

Administration → Configuration → Webhooks



Click on **Create**

Create Webhook

All fields marked with * are required

Name *

jenkins

URL *

http://13.233.17.158:8080/sonarqube-webhook/

Server endpoint that will receive the webhook payload, for example: "http://my_server/foo". If HTTP Basic authentication is used, HTTPS is recommended to avoid man in the middle attacks. Example: "https://myLogin.myPassword@my_server/foo"

Secret

If provided, secret will be used as the key to generate the HMAC hex (lowercase) digest value in the 'X-Sonar-Webhook-HMAC-SHA256' header.

Create **Cancel**

Add details:

COPY

<http://jenkins-private-ip:8080>/sonarqube-webhook/

🌟 5F:Setup new pipeline with name "devsecops-swiggy"

Enter an item name

devsecops-swiggy

Required field

Freestyle project
This is the central feature of Jenkins. Jenkins will build your project, combining any SCM with any build system, and this can be even used for something other than software build.

Pipeline
Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.

Multi-configuration project
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.

Folder
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.

Multibranch Pipeline
Creates a set of Pipeline projects according to detected branches in one SCM repository.

Organization Folder
Creates a set of multibranch project subfolders by scanning for repositories.

OK

Dashboard > devsecops-swiggy > Configuration

Configure

- General
- Advanced Project Options
- Pipeline

☒ Discard old builds ?

Strategy
Log Rotation

Days to keep builds
if not empty, build records are only kept up to this number of days

Max # of builds to keep
if not empty, only up to this number of build records are kept

3

Save Apply

If you want help how to write scripted pipeline, use Pipeline Syntax.

Dashboard > devsecops-swiggy > Pipeline Syntax

Steps

- Steps Reference
- Global Variables Reference
- Online Documentation
- Examples Reference
- IntelliJ IDEA GDSDL

Sample Step
git: Git

git ?

Repository URL ?
https://github.com/dushyantkumark/Swiggy_DevSecOps_Project.git

Branch ?
main

Credentials ?
- none -

+ Add

☒ Include in polling? ?

☒ Include in changelog? ?

Generate Pipeline Script

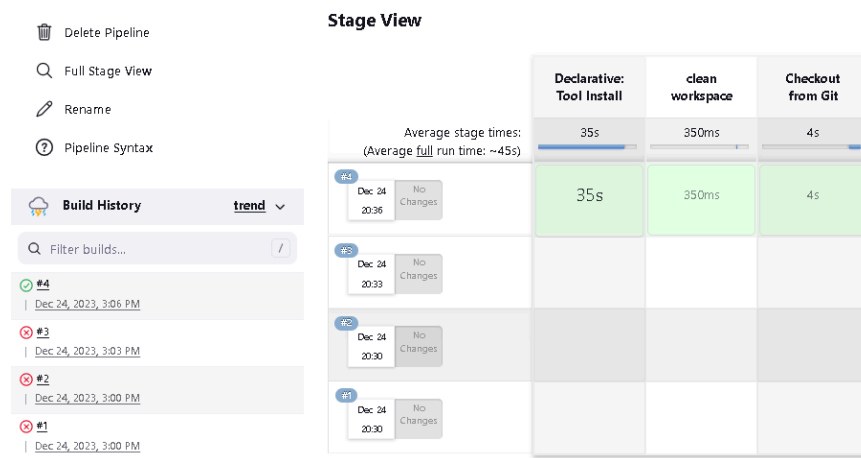
Let's go to our pipeline and add the script to our pipeline script.

COPY

```
pipeline{
    agent any
    tools{
        jdk 'jdk17'
        nodejs 'node16'
    }
    environment {
        SCANNER_HOME=tool 'sonar-scanner'
    }
    stages {
        stage('clean workspace'){
            steps{
                cleanWs()
            }
        }
        stage('Checkout from Git'){
            steps{
                git branch: 'main', url: 'https://github.com/dushyantkumark/S
            }
        }
    }
}
```



Click on Build now, and you will see the stage view like this:



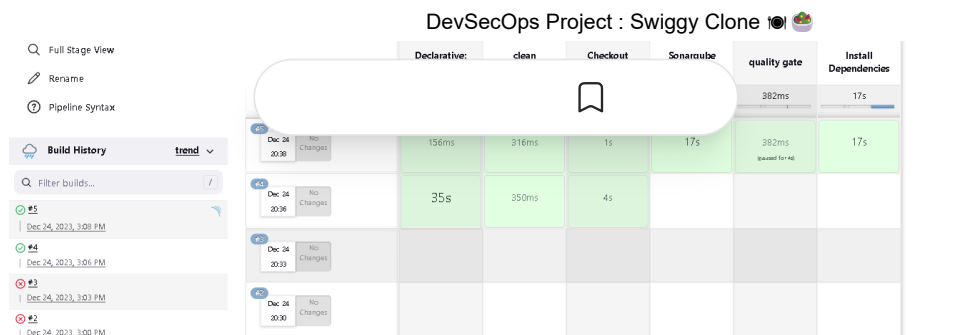
Add more stages such as Sonar analysis, quality gate, install dependencies.

COPY

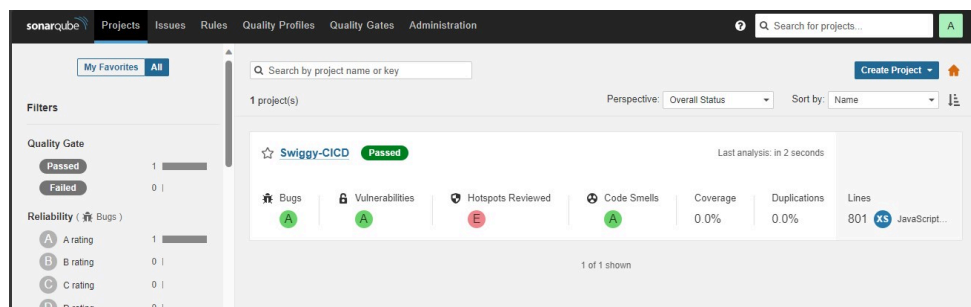
```
stage("Sonarqube Analysis "){
  steps{
    withSonarQubeEnv('sonar-server') {
      sh ''' $SCANNER_HOME/bin/sonar-scanner -Dsonar.projectNam
-Dsonar.projectKey=Swiggy-CICD '''
    }
  }
}
stage("quality gate"){
  steps {
    script {
      waitForQualityGate abortPipeline: false, credentialsId: '
    }
  }
}
stage('Install Dependencies') {
  steps {
    sh "npm install"
  }
}
```



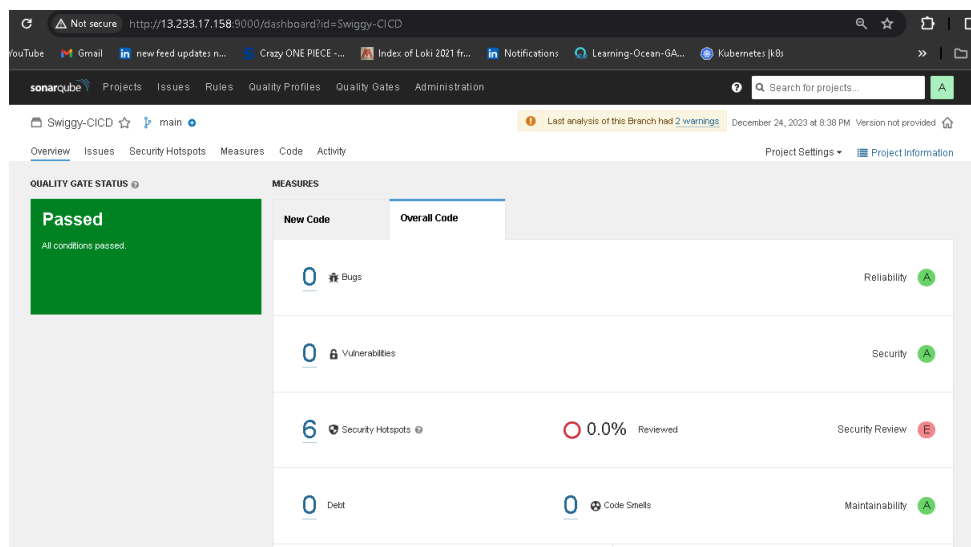
Click on Build now, and you will see the stage view like this:



To see the report, you can go to Sonarqube Server and go to Projects.

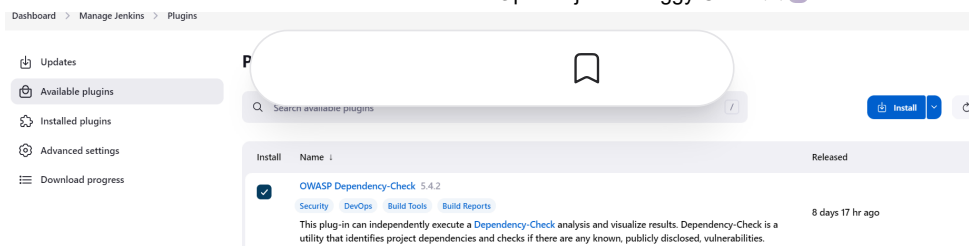


You can see the report has been generated, and the status shows as passed. You can see that there are 801 lines it has scanned. To see a detailed report, you can go to issues.



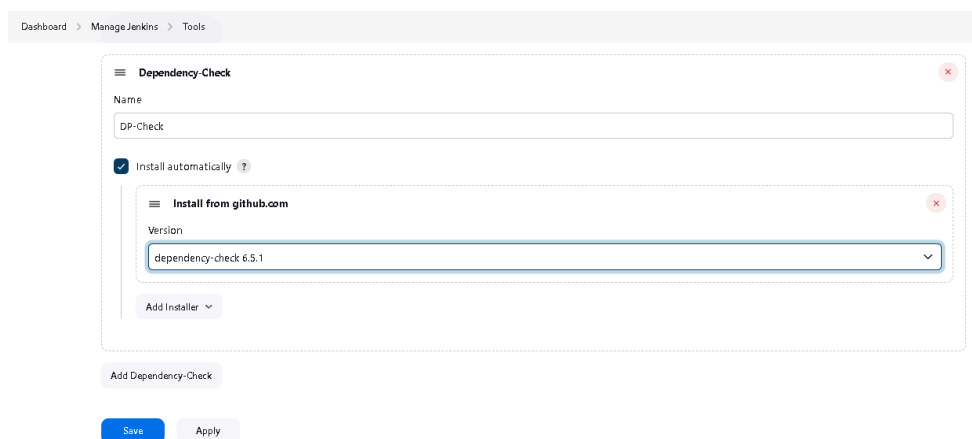
Step 6: Install OWASP Dependency Check Plugins

Go to **Dashboard** → **Manage Jenkins** → **Plugins** → **OWASP Dependency-Check**. Click on it and install it without restarting.



First, we configured the plugin, and next, we had to configure the Tool

Goto **Dashboard** → **Manage Jenkins** → **Tools** →

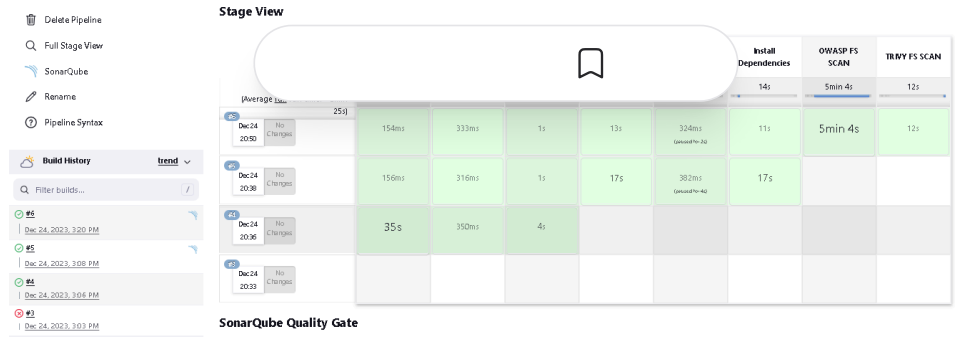


Click on **Apply** and **save** here.

Now go to **Configure** → Pipeline and add this stage to your pipeline and build.



Click on Build now, and you will see the stage view like this:



You will see that in status, a graph will also be generated for vulnerabilities.

Dependency-Check Results

SEVERITY DISTRIBUTION

5	6	6
---	---	---

File Name	Vulnerability	Severity	Weakness
+ axios:1.3.4	OSSINDEX CVE-2023-45857	Medium	CWE-352
+ css:what:3.4.2	OSSINDEX CVE-2022-21222	High	CWE-1333
+ ejs:3.1.9	NVD CVE-2023-29827	Critical	CWE-74
+ jsonpointer:5.0.1	NVD CVE-2022-4742	Critical	CWE-1321
+ nth-check:1.0.2	NVD CVE-2021-3803	High	CWE-1333
+ parseurl:1.3.3	NVD CVE-2022-0722	High	CWE-200
+ parseurl:1.3.3	NVD CVE-2022-2216	Critical	CWE-918
+ parseurl:1.3.3	NVD CVE-2022-2217	Medium	CWE-79
+ parseurl:1.3.3	NVD CVE-2022-2218	Medium	CWE-79
+ parseurl:1.3.3	NVD CVE-2022-2900	Critical	CWE-918

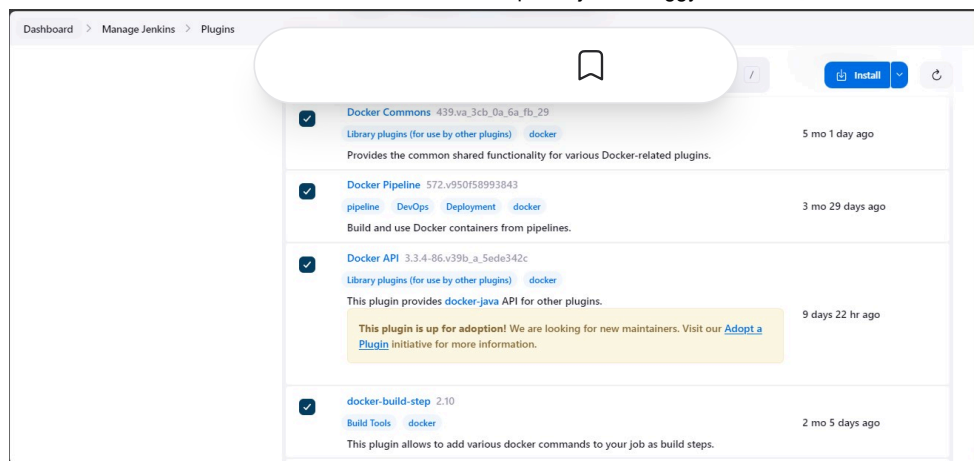
Step 7: Docker Image Build and Push

We need to install the Docker tool on our system.

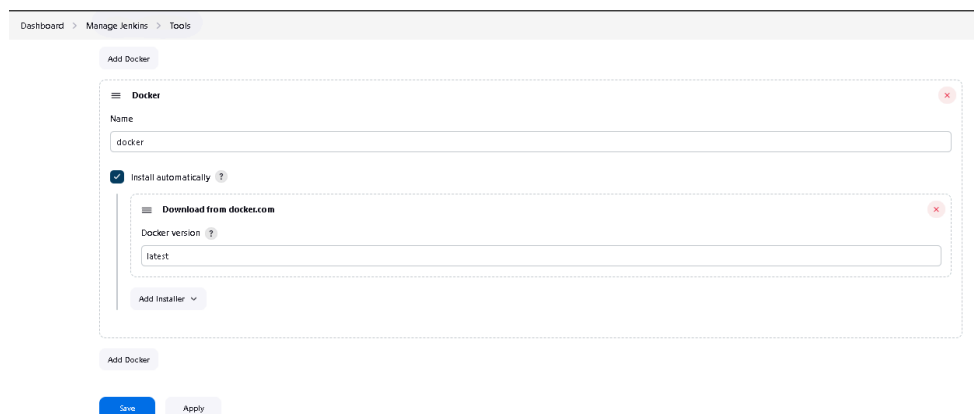
Go to **Dashboard** → **Manage Plugins** → **Available plugins** → Search for **Docker** and install these plugins.

Install these plugins:

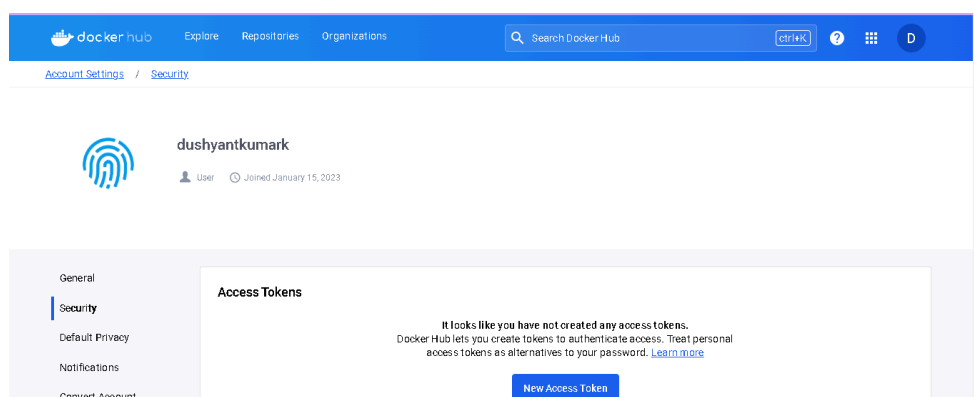
- Docker
- Docker Commons
- Docker Pipeline
- Docker API Docker-build-step



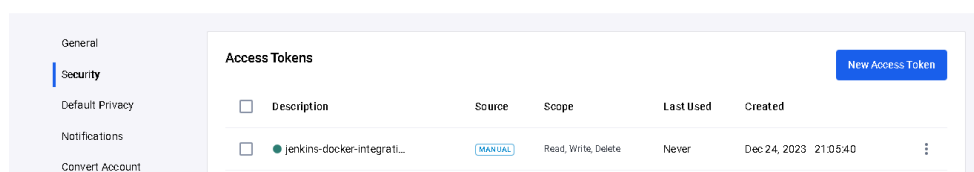
Now, goto **Dashboard** → **Manage Jenkins** → **Tools** →



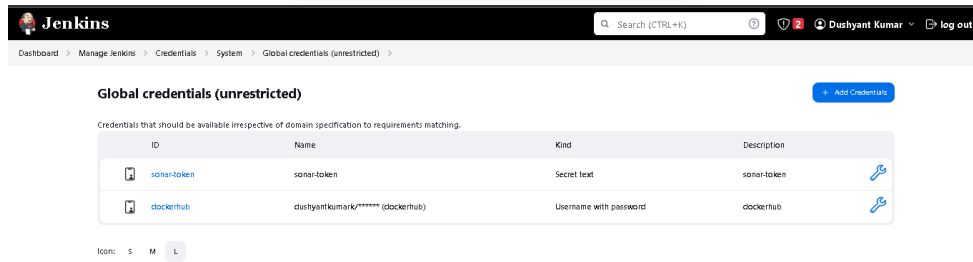
Now go to the Dockerhub repository to generate a token and integrate with Jenkins to push the image to the specific repository.



Click on that **My Account**, → **Security** → **New access token** and copy the token.



Goto Jenkins Dashboard → Manage Jenkins → Credentials → Add secret text.
It should look like this

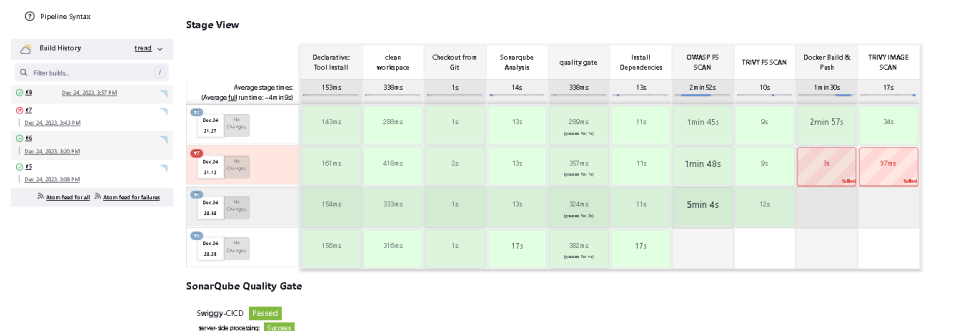


Add this stage to Pipeline Script.

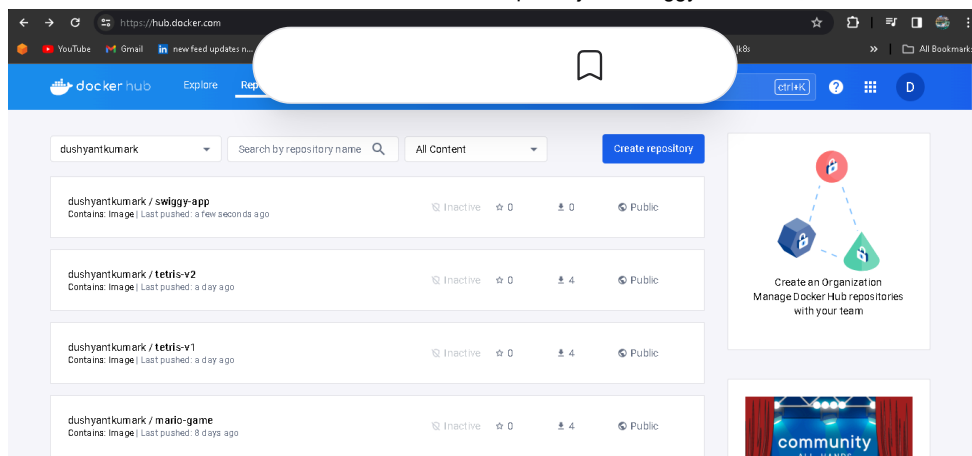
COPY

```
stage("Docker Build & Push"){
    steps{
        script{
            withDockerRegistry(credentialsId: 'dockerhub', toolName: 'docker'){
                app.push("${env.BUILD_NUMBER}")
                sh "docker build -t swiggy-app ."
                sh "docker tag swiggy-app dushyantkumar/swiggy-app:latest"
                sh "docker push dushyantkumar/swiggy-app:latest"
            }
        }
    }
}

stage("TRIVY IMAGE SCAN"){
    steps{
        sh "trivy image dushyantkumar/swiggy-app:latest > trivyimage"
    }
}
```



When you log in to Dockerhub, you will see a new image is created.



Now test the swiggy application is running or not.

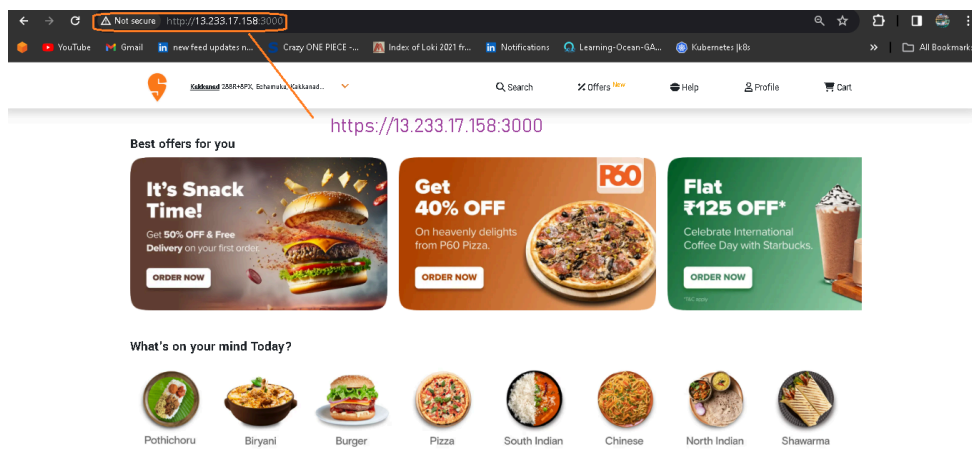
Add this stage to Pipeline Script.

COPY

```
stage("Deploy to container"){
    steps{
        sh "docker run -d -p 3000:80 --name swiggy dushyantkumark/swi"
    }
}
```



Now access your application.



Step 8: Creation of EKS Cluster with ArgoCD

Installation of KUBECTL, EKSCTL, AWS CLI are done at the time of ec2 instance creation using terraform.

8A: Creation of EKS Cluster

Now let's configure aws cli credentials for eks creation in us-east-1 region.

COPY

aws configure

add accessKey, secretAccessKey, region, output format.



Command to Create EKS Cluster using eksctl command:

COPY

```
eksctl create cluster --name <name-of-cluster> --region <regionName> --node-t
```

```
eksctl create cluster --name swiggy-cluster --region us-east-1 --node-type t2
```



```

ubuntu@ip-172-31-32-108:~$ eksctl create cluster --name swiggy-cluster --region us-east-1 --node-type t2.medium --nodes-min 2 --nodes-max 4
2023-12-24 16:39:44 [i] eksctl version 0.167.0
2023-12-24 16:39:44 [i] using region us-east-1
2023-12-24 16:39:45 [i] setting availability zones to [us-east-1c us-east-1a]
2023-12-24 16:39:45 [i] subnets for us-east-1c - public:192.168.0.0/19 private:192.168.64.0/19
2023-12-24 16:39:45 [i] subnets for us-east-1a - public:192.168.32.0/19 private:192.168.96.0/19
2023-12-24 16:39:45 [i] nodegroup "ng-c7362b7c" will use "" [AmazonLinux2/1.27]
2023-12-24 16:39:45 [i] using Kubernetes version 1.27
2023-12-24 16:39:45 [i] creating EKS cluster "swiggy-cluster" in "us-east-1" region with managed nodes
2023-12-24 16:39:45 [i] will create 2 separate CloudFormation stacks for cluster itself and the initial managed nodegroup
2023-12-24 16:39:45 [i] if you encounter any issues, check CloudFormation console or try 'eksctl utils describe-stacks --region=us-east-1 --clu
ster=swiggy-cluster'
2023-12-24 16:39:45 [i] Kubernetes API endpoint access will use default of {publicAccess=true, privateAccess=false} for cluster "swiggy-cluster"
in "us-east-1"
2023-12-24 16:39:45 [i] CloudWatch logging will not be enabled for cluster "swiggy-cluster" in "us-east-1"
2023-12-24 16:39:45 [i] you can enable it with 'eksctl utils update-cluster-logging --enable-types={SPECIFY-YOUR-LOG-TYPES-HERE (e.g. all)} --r
egion=us-east-1 --cluster=swiggy-cluster'
2023-12-24 16:39:45 [i]
2 sequential tasks: { create cluster control plane "swiggy-cluster",
  2 sequential sub-tasks: {
    wait for control plane to become ready,
    create managed nodegroup "ng-c7362b7c",
  }
}
2023-12-24 16:39:45 [i] building cluster stack "eksctl-swiggy-cluster-cluster"
2023-12-24 16:39:46 [i] deploying stack "eksctl-swiggy-cluster-cluster"

```

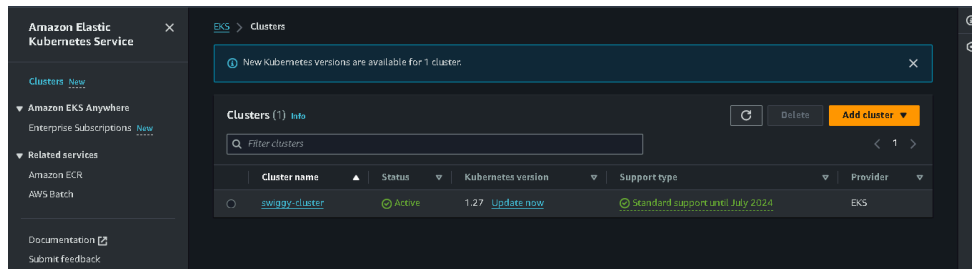
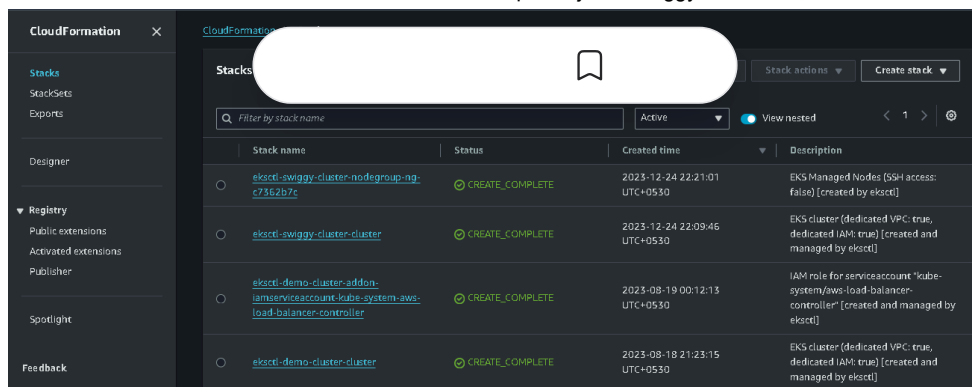
It will take 10-15 minutes to create a cluster.

```

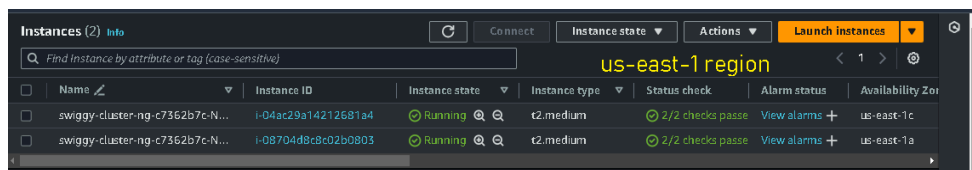
2023-12-24 16:40:16 [i] waiting for CloudFormation stack "eksctl-swiggy-cluster-cluster"
2023-12-24 16:40:47 [i] waiting for CloudFormation stack "eksctl-swiggy-cluster-cluster"
2023-12-24 16:41:48 [i] waiting for CloudFormation stack "eksctl-swiggy-cluster-cluster"
2023-12-24 16:42:48 [i] waiting for CloudFormation stack "eksctl-swiggy-cluster-cluster"
2023-12-24 16:43:49 [i] waiting for CloudFormation stack "eksctl-swiggy-cluster-cluster"
2023-12-24 16:44:50 [i] waiting for CloudFormation stack "eksctl-swiggy-cluster-cluster"
2023-12-24 16:45:51 [i] waiting for CloudFormation stack "eksctl-swiggy-cluster-cluster"
2023-12-24 16:46:51 [i] waiting for CloudFormation stack "eksctl-swiggy-cluster-cluster"
2023-12-24 16:47:52 [i] waiting for CloudFormation stack "eksctl-swiggy-cluster-cluster"
2023-12-24 16:48:53 [i] waiting for CloudFormation stack "eksctl-swiggy-cluster-cluster"
2023-12-24 16:51:09 [i] building managed nodegroup stack "eksctl-swiggy-cluster-nodegroup-ng-c7362b7c"
2023-12-24 16:51:01 [i] deploying stack "eksctl-swiggy-cluster-nodegroup-ng-c7362b7c"
2023-12-24 16:51:01 [i] waiting for CloudFormation stack "eksctl-swiggy-cluster-nodegroup-ng-c7362b7c"
2023-12-24 16:51:32 [i] waiting for CloudFormation stack "eksctl-swiggy-cluster-nodegroup-ng-c7362b7c"
2023-12-24 16:52:28 [i] waiting for CloudFormation stack "eksctl-swiggy-cluster-nodegroup-ng-c7362b7c"
2023-12-24 16:54:04 [i] waiting for CloudFormation stack "eksctl-swiggy-cluster-nodegroup-ng-c7362b7c"
2023-12-24 16:54:43 [i] waiting for CloudFormation stack "eksctl-swiggy-cluster-nodegroup-ng-c7362b7c"
2023-12-24 16:55:23 [i] waiting for CloudFormation stack "eksctl-swiggy-cluster-nodegroup-ng-c7362b7c"
2023-12-24 16:55:23 [i] waiting for the control plane to become ready
2023-12-24 16:55:24 [x] saved kubeconfig as "/home/ubuntu/.kube/config"
2023-12-24 16:55:24 [i] no tasks
2023-12-24 16:55:24 [x] all EKS cluster resources for "swiggy-cluster" have been created
2023-12-24 16:55:25 [i] nodegroup "ng-c7362b7c" has 2 node(s)
2023-12-24 16:55:25 [i] node "ip-192-168-22-108.ec2.internal" is ready
2023-12-24 16:55:25 [i] node "ip-192-168-50-209.ec2.internal" is ready
2023-12-24 16:55:25 [i] waiting for at least 2 node(s) to become ready in "ng-c7362b7c"
2023-12-24 16:55:25 [i] nodegroup "ng-c7362b7c" has 2 node(s)
2023-12-24 16:55:25 [i] node "ip-192-168-22-108.ec2.internal" is ready
2023-12-24 16:55:25 [i] node "ip-192-168-50-209.ec2.internal" is ready
2023-12-24 16:55:26 [i] kubectl command should work with "/home/ubuntu/.kube/config", try 'kubectl get nodes'
2023-12-24 16:55:26 [x] EKS cluster "swiggy-cluster" in "us-east-1" region is ready
ubuntu@ip-172-31-32-108:~$

```

Check your cloudformation stack and ekscluster.



As you will see in the EC2 instances nodes running in the name of EKS Cluster (swiggy-cluster) as shown below.



EKS Cluster is up and ready and check with the below command.

COPY

```
kubectl get no
kubectl get po
```

8B: Installing ArgoCD

Now let's install ArgoCD in the EKS Cluster.

COPY

```
kubectl create ns argocd
# This will create a new namespace, argocd, where Argo CD services and applic
kubectl create namespace argocd
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd
```

```

ubuntu@ip-172-31-50-204:~$ kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
customresourcedefinition.apiextensions.k8s.io/customresourcedefinitions created
serviceaccount/argocd-application-controller created
serviceaccount/argocd-applicationset-controller created
serviceaccount/argocd-dex-server created
serviceaccount/argocd-notifications-controller created
serviceaccount/argocd-redis created
serviceaccount/argocd-repo-server created
serviceaccount/argocd-server created
role.rbac.authorization.k8s.io/argocd-application-controller created
role.rbac.authorization.k8s.io/argocd-applicationset-controller created
role.rbac.authorization.k8s.io/argocd-dex-server created
role.rbac.authorization.k8s.io/argocd-notifications-controller created
role.rbac.authorization.k8s.io/argocd-server created
clusterrole.rbac.authorization.k8s.io/argocd-application-controller unchanged
clusterrole.rbac.authorization.k8s.io/argocd-server unchanged
rolebinding.rbac.authorization.k8s.io/argocd-application-controller created
rolebinding.rbac.authorization.k8s.io/argocd-applicationset-controller created
rolebinding.rbac.authorization.k8s.io/argocd-dex-server created

```

Download Argo CD CLI:

COPY

```

curl -sSL -o argocd-linux-amd64 https://github.com/argoproj/argo-cd/releases/
sudo install -m 555 argocd-linux-amd64 /usr/local/bin/argocd

```

Access The Argo CD API Server:

COPY

```

# By default, the Argo CD API server is not exposed with an external IP. To a
choose one of the following techniques to expose the Argo CD API server:
* Service Type Load Balancer
* Port Forwarding

```

Let's go with Service Type Load Balancer.

COPY

```

# Change the argocd-server service type to LoadBalancer.
kubectl patch svc argocd-server -n argocd -p '{"spec": {"type": "LoadBalancer"

```

Wait about 2 minutes for the LoadBalancer creation

COPY

```

export ARGOCD_SERVER=$(kubectl get svc argocd-server -n argocd -o json | jq --r

```

The initial password is autogenerated with the pod name of the ArgoCD API server.

COPY

```

export ARGO_PWD=$(kubectl -n argocd get secret argocd-initial-admin-secret -o

```

NOW get the loadbalancer



COPY

```
echo $ARGOCD_SERVER
echo $ARGO_PWD
```

```
ubuntu@ip-172-31-32-108:~$ export ARGOCD_SERVER=$(kubectl get svc argocd-server -n argocd -o json | jq --raw-output '.status.loadBalancer.ingress[0].hostname')
Kubeconfig user entry is using deprecated API version client.authentication.k8s.io/v1alpha1. Run 'aws eks update-kubeconfig' to update.
ubuntu@ip-172-31-32-108:~$ export ARGO_PWD=$(kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath='{.data.password}' | base64 -d)
Kubeconfig user entry is using deprecated API version client.authentication.k8s.io/v1alpha1. Run 'aws eks update-kubeconfig' to update.
ubuntu@ip-172-31-32-108:~$ echo $ARGOCD_SERVER
adf0130b35ba44ecfb2ec1b692b87177-588955684.us-east-1.elb.amazonaws.com
ubuntu@ip-172-31-32-108:~$ echo $ARGO_PWD
b0ETU5mfs431owR
ubuntu@ip-172-31-32-108:~$
```

List the resources in the namespace:

COPY

```
kubectl get all -n argocd
```

```
ubuntu@ip-172-31-32-108:~$ kubectl get all -n argocd
NAME                                READY     UP-TO-DATE     AVAILABLE     AGE
deployment.apps/argocd-applicationset-controller  1/1       1               1             13m
deployment.apps/argocd-dex-server                1/1       1               1             13m
deployment.apps/argocd-notifications-controller  1/1       1               1             13m
deployment.apps/argocd-redis                    1/1       1               1             13m
deployment.apps/argocd-repo-server              1/1       1               1             13m
deployment.apps/argocd-server                   1/1       1               1             13m
service/argocd-applicationset-controller         ClusterIP  10.100.151.64   <none>        13m
service/argocd-dex-server                       ClusterIP  10.100.142.93   <none>        13m
service/argocd-notifications-controller-metrics ClusterIP  10.100.88.169   <none>        13m
service/argocd-redis                           ClusterIP  10.100.173.97   <none>        13m
service/argocd-repo-server                     ClusterIP  10.100.38.235   <none>        13m
service/argocd-server                          ClusterIP  10.100.102.71   <none>        13m
service/argocd-server-metrics                  LoadBalancer 10.100.133.73   adf0130b35ba44ecfb2ec1b692b87177-588955684.us-east-1.elb.amazonaws.com 80:30360
service/argocd-server-metrics                  ClusterIP  10.100.121.29   <none>        13m
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/argocd-applicationset-controller	1/1	1	1	13m
deployment.apps/argocd-dex-server	1/1	1	1	13m
deployment.apps/argocd-notifications-controller	1/1	1	1	13m
deployment.apps/argocd-redis	1/1	1	1	13m
deployment.apps/argocd-repo-server	1/1	1	1	13m
deployment.apps/argocd-server	1/1	1	1	13m

NAME	READY	DESIRED	CURRENT	READY	AGE
replicaset.apps/argocd-applicationset-controller-dc5c4965	1	1	1	1	13m
replicaset.apps/argocd-dex-server-9769d699	1	1	1	1	13m
replicaset.apps/argocd-notifications-controller-db4f975f8	1	1	1	1	13m
replicaset.apps/argocd-redis-b5d0bf5f5	1	1	1	1	13m
replicaset.apps/argocd-repo-server-979dc7849	1	1	1	1	13m
replicaset.apps/argocd-server-557c4d6ff	1	1	1	1	13m

NAME	READY	AGE
statefulset.apps/argocd-application-controller	1/1	13m

Get the load balancer URL:

COPY

```
kubectl get svc -n argocd
```

```
ubuntu@ip-172-31-32-108:~$ kubectl get svc -n argocd
NAME                                TYPE                CLUSTER-IP      EXTERNAL-IP      PORT(S)
argocd-applicationset-controller    ClusterIP           10.100.151.64    <none>            7000/TCP, 8080/TCP
argocd-dex-server                   ClusterIP           10.100.142.93    <none>            5556/TCP, 5557/TCP
argocd-notifications-controller-metrics ClusterIP           10.100.88.169    <none>            8082/TCP
argocd-redis                       ClusterIP           10.100.173.97    <none>            9001/TCP
argocd-repo-server                 ClusterIP           10.100.38.235    <none>            6379/TCP
argocd-server                      ClusterIP           10.100.102.71    <none>            8081/TCP, 8084/TCP
argocd-server-metrics              LoadBalancer        10.100.133.73    adf0130b35ba44ecfb2ec1b692b87177-588955684.us-east-1.elb.amazonaws.com 80:30360/TCP, 443
argocd-server-metrics              ClusterIP           10.100.121.29    <none>            8083/TCP
```

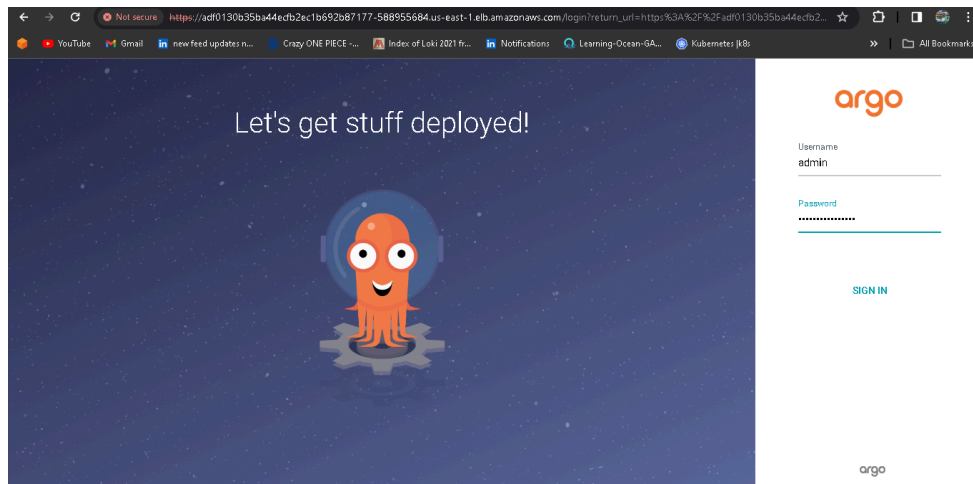
Lets create new namespace for application.

COPY

```
kubectl create ns
```



Pickup the URL and paste it into the web to get the UI as shown below image:



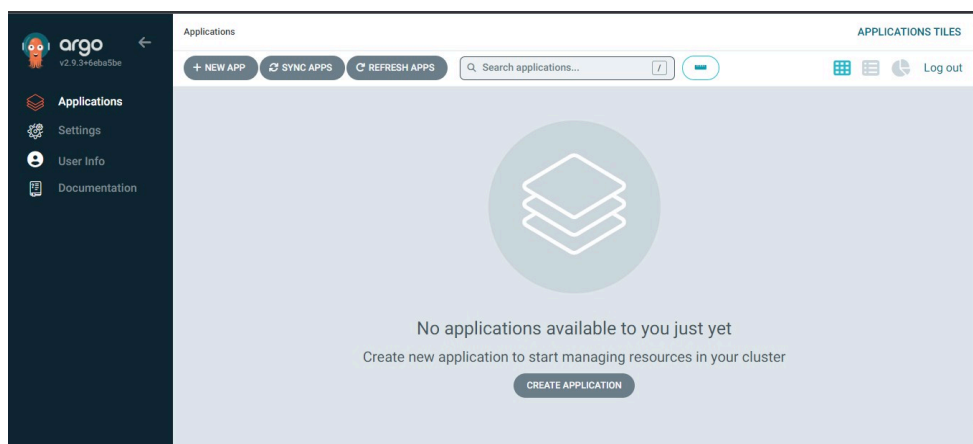
Login Using ARGO_PWD***

COPY

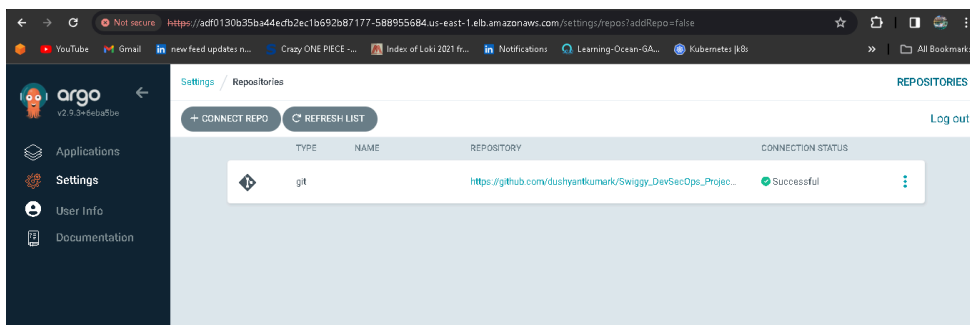
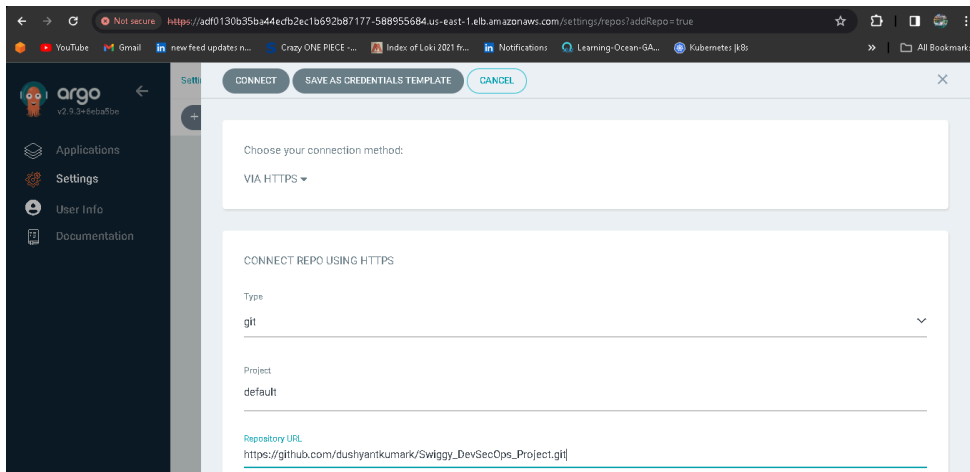
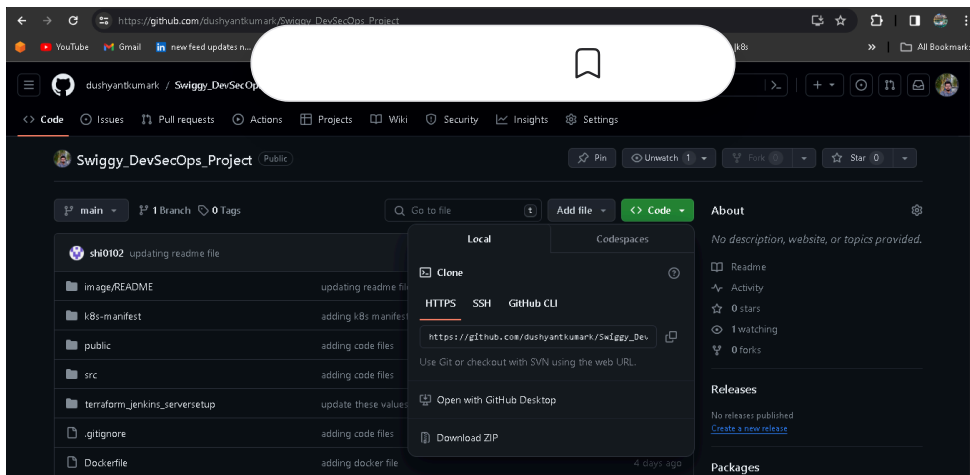
```
echo ARGO_PWD
```

```
ubuntu@ip-172-31-32-108:~$ export ARGOCD_SERVER= kubectl get svc argocd-server -n argocd -o json | jq --raw-output '.status.loadBalancer.ingress[0].hostname'
Kubeconfig user entry is using deprecated API version client.authentication.k8s.io/v1alpha1. Run 'aws eks update-kubeconfig' to update.
ubuntu@ip-172-31-32-108:~$ export ARGO_PWD= kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath='{.data.password}' | base64 -d
Kubeconfig user entry is using deprecated API version client.authentication.k8s.io/v1alpha1. Run 'aws eks update-kubeconfig' to update.
ubuntu@ip-172-31-32-108:~$ echo $ARGOCD_SERVER
adf0130b35ba44ecb2ec1b692b87177-588955684.us-east-1.elb.amazonaws.com
ubuntu@ip-172-31-32-108:~$ echo $ARGO_PWD
bbETTUSmfs4JiomR
ubuntu@ip-172-31-32-108:~$
```

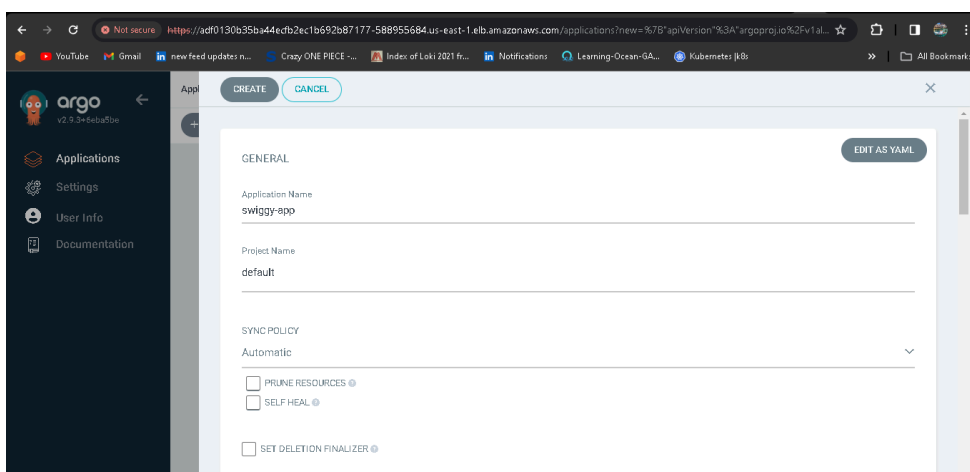
Login with the admin and Password in the above you will get an interface as shown below:



Lets connect with github repository:



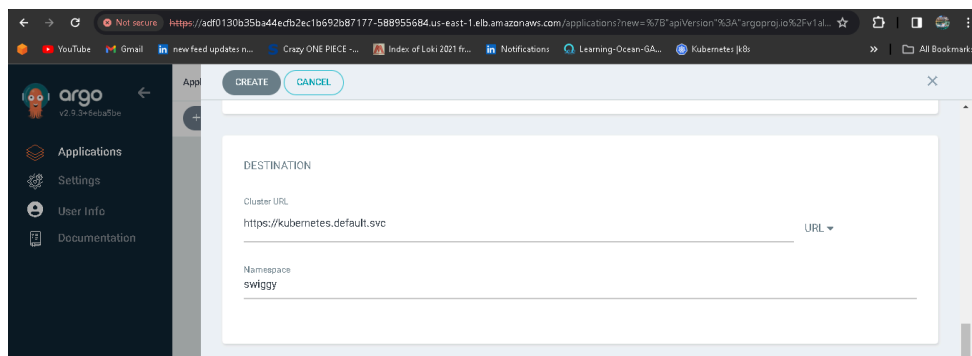
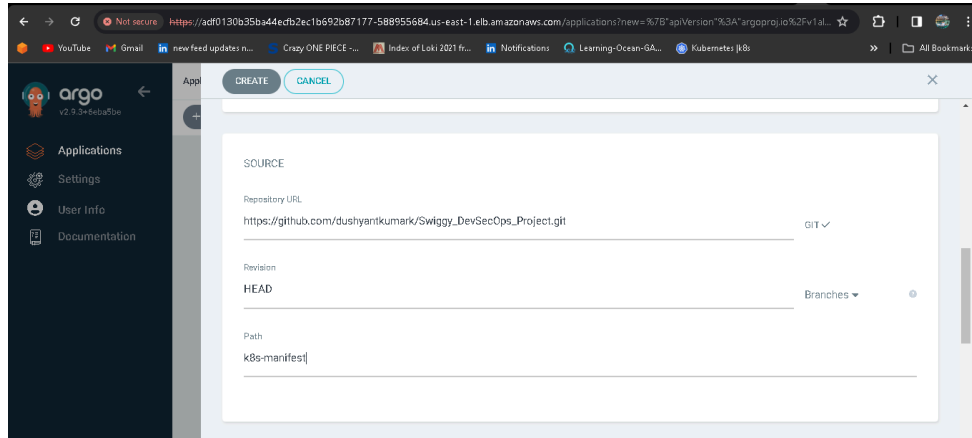
Now deploy an application, first we have to configure the details in Application.



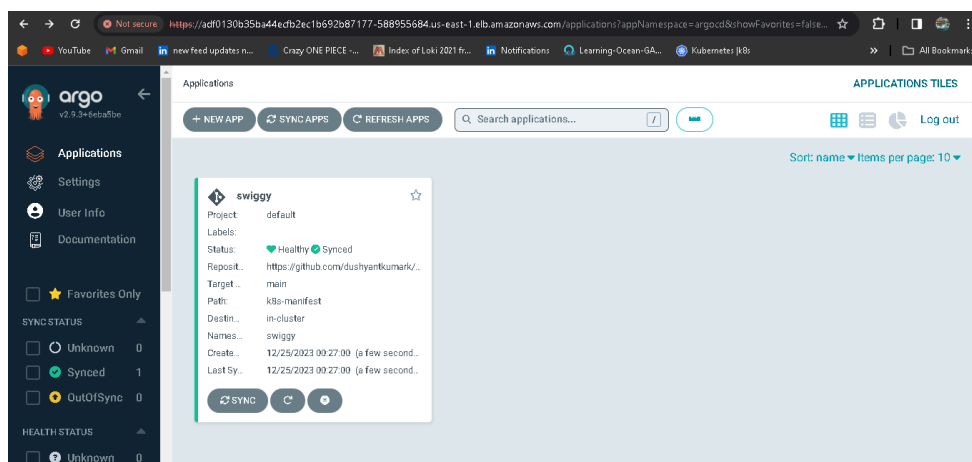
Enter the Repository URL, which is `https://github.com/dushyantkumark/Swiggy_DevSecOps_Project.git`, the namespace to `swiggy`.

The GitHub URL is the Kubernetes Manifest files which I have stored and the pushed image is used in the Kubernetes deployment files.

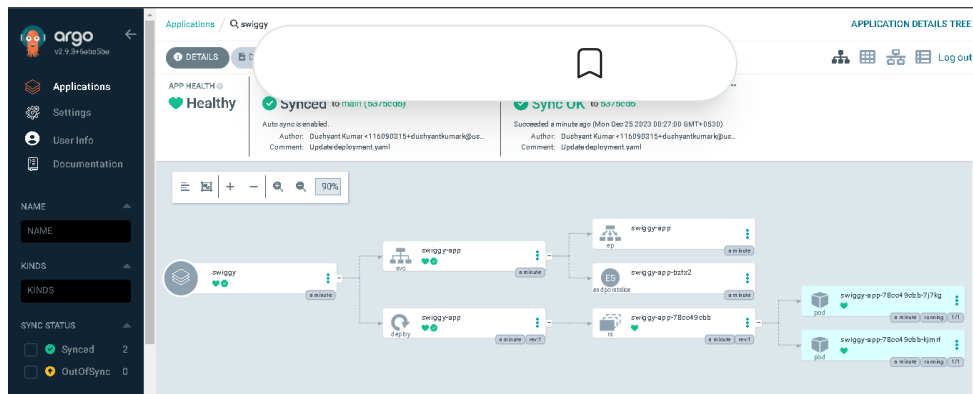
Repo Link: https://github.com/dushyantkumark/Swiggy_DevSecOps_Project.git



You should see the below, once you're done with the details.



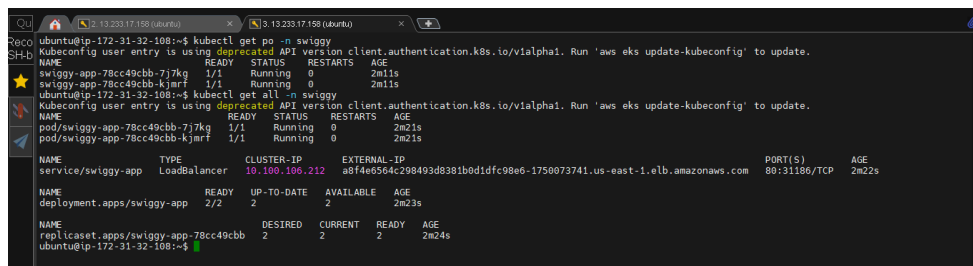
Click on it.



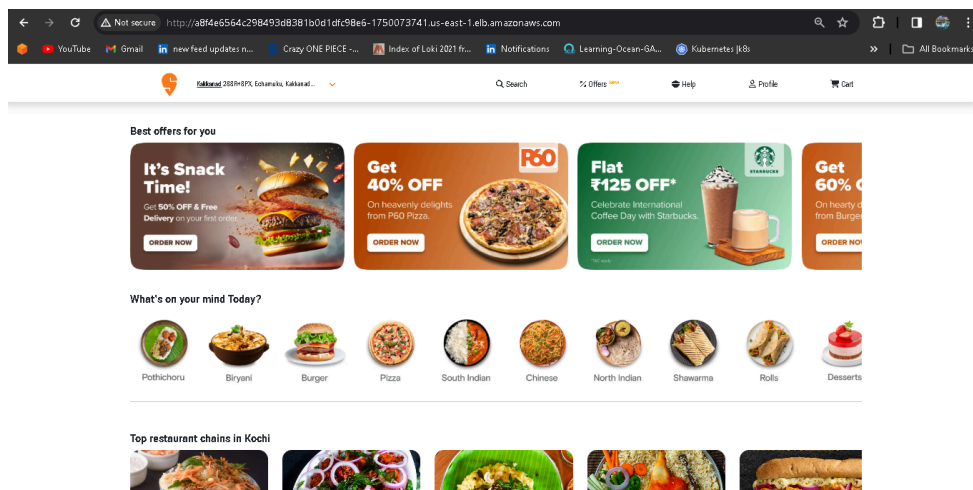
You can see the pods running in the EKS Cluster.

COPY

```
kubectl get po -n swiggy
kubectl get all -n swiggy
```



With the above load balancer, you will be able to see the output as shown in the below image:



🔥 Step 9: Clean up your resources:

- Delete your Cluster

COPY

```
eksctl delete cluster --name swiggy-cluster
```

```

ubuntu@ip-172-31-32-108:~$ eksctl delete cluster --name swiggy-cluster
2023-12-24 19:18:52 [i] deleting EKS cluster "swiggy-cluster"
2023-12-24 19:18:54 [i] will delete stack "eksctl-swiggy-cluster-nodegroup-ng-c7362b7c"
2023-12-24 19:18:55 [i] starting deletion of stack "eksctl-swiggy-cluster-nodegroup-ng-c7362b7c"
2023-12-24 19:18:57 [✓] kubeconfig file "swiggy-cluster-admin" deleted
2023-12-24 19:18:57 [i] cleaning up AWS load balancers created by Kubernetes objects of Kind Service or Ingress
2023-12-24 19:19:26 [i] 2 sequential tasks: { delete nodegroup "ng-c7362b7c", delete cluster control plane "swiggy-cluster" [async]
}
2023-12-24 19:19:27 [i] will delete stack "eksctl-swiggy-cluster-nodegroup-ng-c7362b7c"
2023-12-24 19:19:27 [i] waiting for stack "eksctl-swiggy-cluster-nodegroup-ng-c7362b7c" to get deleted
2023-12-24 19:19:27 [i] waiting for CloudFormation stack "eksctl-swiggy-cluster-nodegroup-ng-c7362b7c"
2023-12-24 19:19:58 [i] waiting for CloudFormation stack "eksctl-swiggy-cluster-nodegroup-ng-c7362b7c"
2023-12-24 19:20:34 [i] waiting for CloudFormation stack "eksctl-swiggy-cluster-nodegroup-ng-c7362b7c"
2023-12-24 19:22:28 [i] waiting for CloudFormation stack "eksctl-swiggy-cluster-nodegroup-ng-c7362b7c"
2023-12-24 19:23:33 [i] waiting for CloudFormation stack "eksctl-swiggy-cluster-nodegroup-ng-c7362b7c"
2023-12-24 19:25:16 [i] waiting for CloudFormation stack "eksctl-swiggy-cluster-nodegroup-ng-c7362b7c"
2023-12-24 19:26:35 [i] waiting for CloudFormation stack "eksctl-swiggy-cluster-nodegroup-ng-c7362b7c"
2023-12-24 19:28:08 [i] waiting for CloudFormation stack "eksctl-swiggy-cluster-nodegroup-ng-c7362b7c"
2023-12-24 19:30:01 [i] waiting for CloudFormation stack "eksctl-swiggy-cluster-nodegroup-ng-c7362b7c"
2023-12-24 19:30:02 [i] will delete stack "eksctl-swiggy-cluster-cluster"
2023-12-24 19:30:03 [✓] all cluster resources were deleted
ubuntu@ip-172-31-32-108:~$

```

- Destroy your Infrastructure using terraform from your laptop (windows)

COPY

terraform destroy

```

- from_port      = 9000
- ipv6_cidr_blocks = []
- prefix_list_ids = []
- protocol       = "tcp"
- security_groups = []
- self           = false
- to_port        = 9000
},
] -> null
- name           = "jenkins-sg" -> null
- owner_id       = "471018761652" -> null
- revoke_rules_on_delete = false -> null
- tags           = {
  - "Name" = "jenkins-security-group"
} -> null
- tags_all       = {
  - "Name" = "jenkins-security-group"
} -> null
- vpc_id         = "vpc-0aef54445ced2dc34" -> null
}

Plan: 0 to add, 0 to change, 5 to destroy.

Do you really want to destroy all resources?
Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes

```

- Delete IAM credentials you create, remove policies from users.

Conclusion:

In this project, we have covered the essential steps to deploy a Swiggy app with a strong focus on security through a DevSecOps approach. By leveraging tools like Terraform, Jenkins CI/CD, SonarQube, Trivy, Argocd, and EKS, we can create a robust and secure pipeline for deploying applications on AWS.

Remember that security is an ongoing process, and it is crucial to stay updated with the latest security practices and continuously monitor and improve the security of your applications. With the knowledge gained from this guide, you can enhance your DevSecOps skills and ensure the smooth and efficient deployment of secure applications on Amazon Web Services.

.....

The above information is up to my understanding. Suggestions are always welcome. Thanks for reading this article.

#docker #aws #cloudnative #devsecops
#sonarqube #trivy #devopsc
#happylearning

Follow for many such contents:

LinkedIn: [linkedin.com/in/dushyant-kumar-dk](https://www.linkedin.com/in/dushyant-kumar-dk)

Blog: dushyantkumark.hashnode.dev

Github: https://github.com/dushyantkumark/Swiggy_DevSecOps_Project.git

Subscribe to our newsletter

Read articles from directly inside your inbox. Subscribe to the newsletter, and don't miss out.

SUBSCRIBE

DevSecOps

Devops

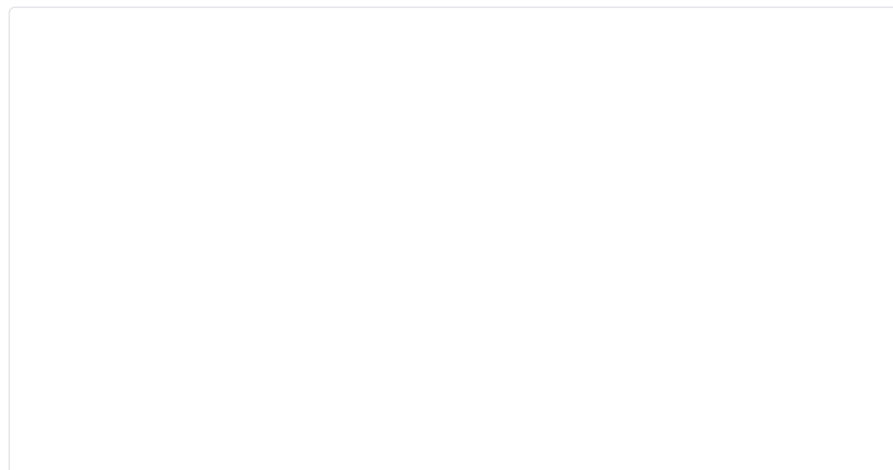
#90daysofdevops

TrainWithShubham

Cloud Computing

MORE ARTICLES

Dushyant Kumar



How to Pull Images from ECR to Private EC2 Without Internet

Instead of using a NAT gateway to connect to the internet, you can set up VPC endpoints to



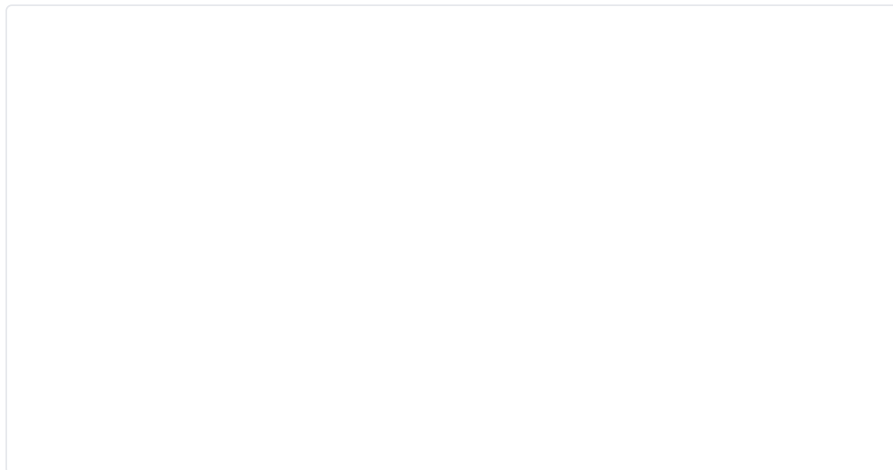
Dushyant Kumar



From VPN to Docker: Securely Connecting AWS Private Instances

Requirements VPC (Virtual Private Cloud): VPC is a virtual network dedicated to your AWS account. I...

Dushyant Kumar



Step-By-Step Guide to Creating a Custom Virtual Private Cloud

Introduction: In the realm of cloud computing, Amazon Web Services (AWS) stands out as a powerhouse,...



[Archive](#) • [Privacy policy](#) • [Terms](#)



Powered by Hashnode - Build your developer hub.

[Start your blog](#)

[Create docs](#)